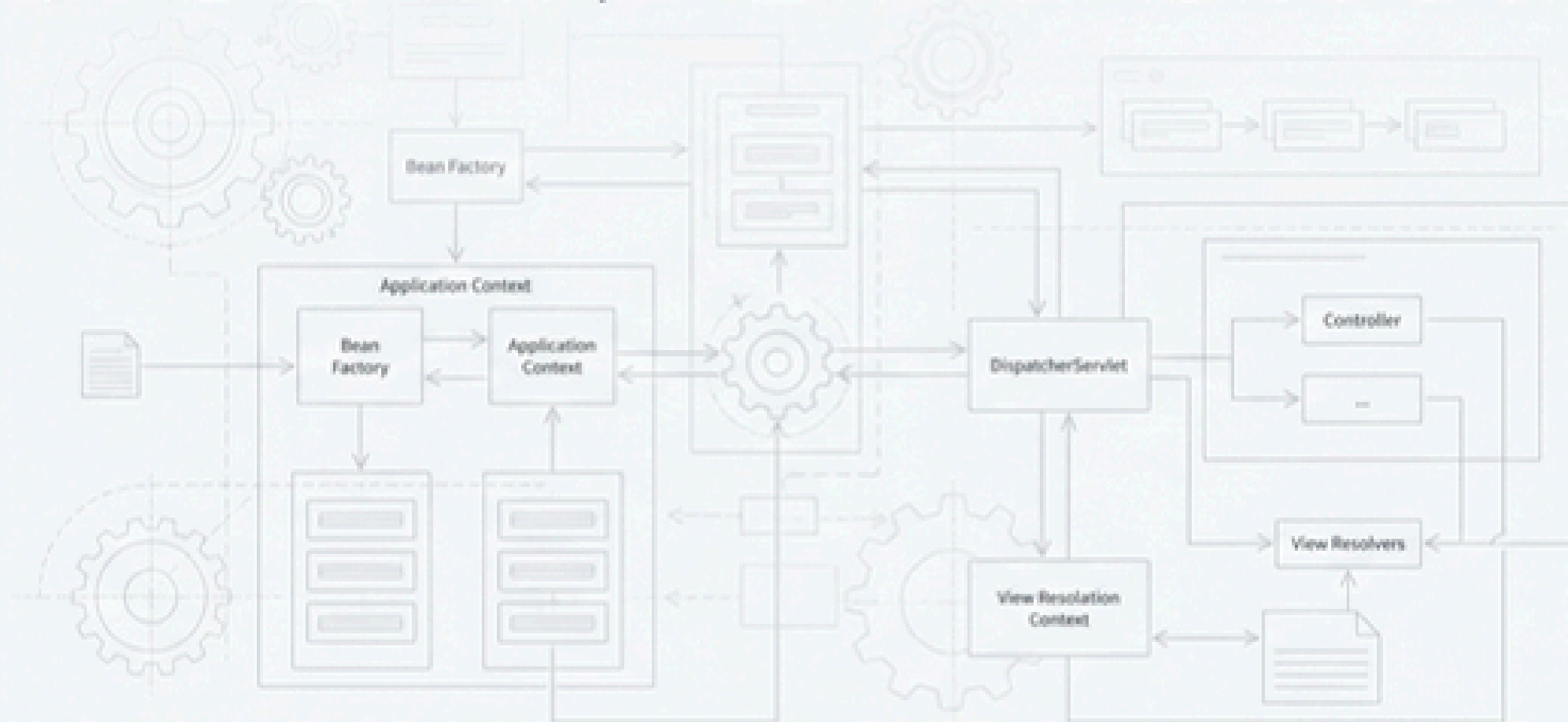


Spring의 심장: Core와 Web Web MVC 동작 원리 파헤치기

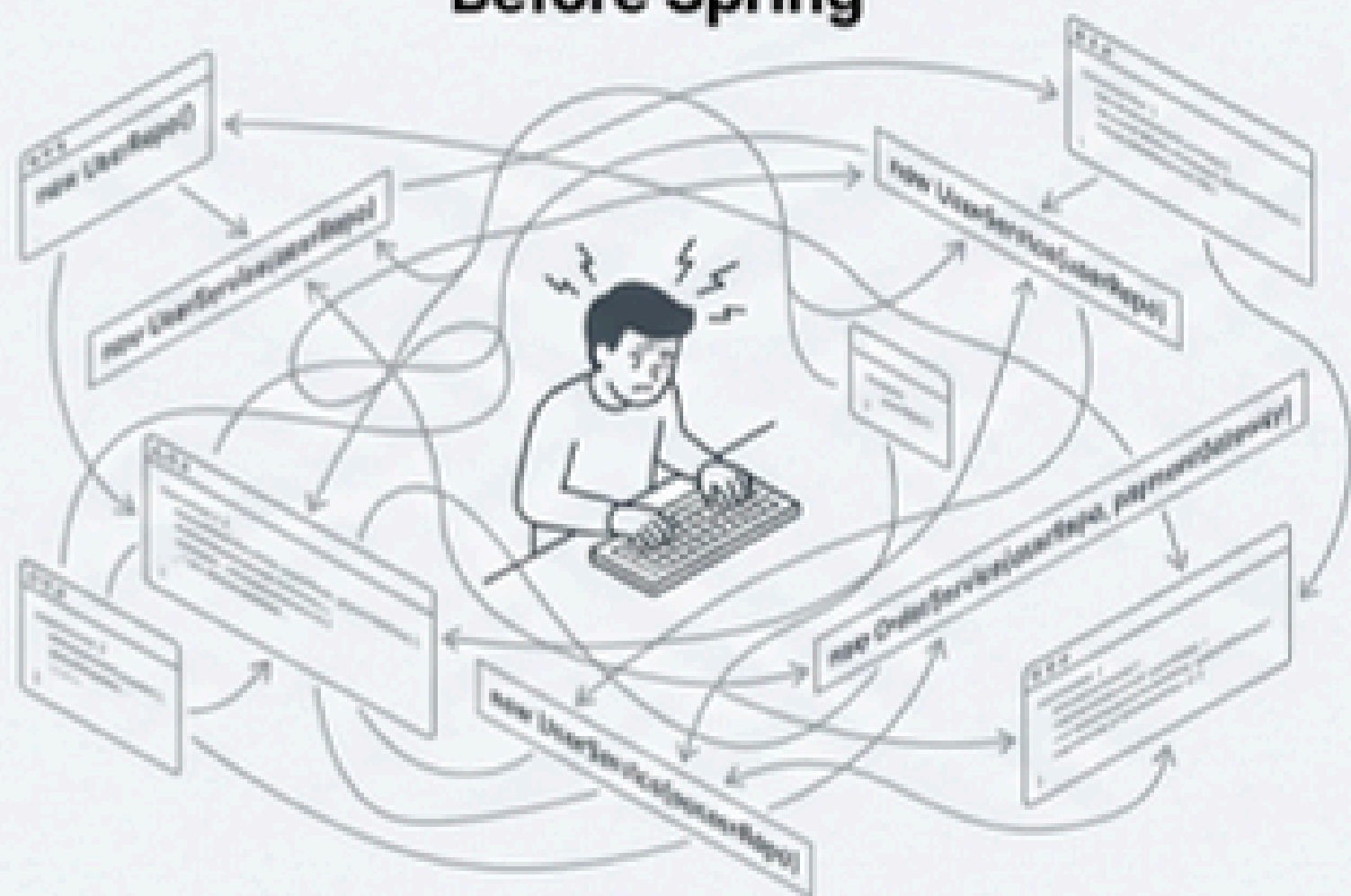
스프링의 핵심 원리를 그림으로 이해하고, 코드 레벨에서 어노테이션의 동작을 마스터합니다.



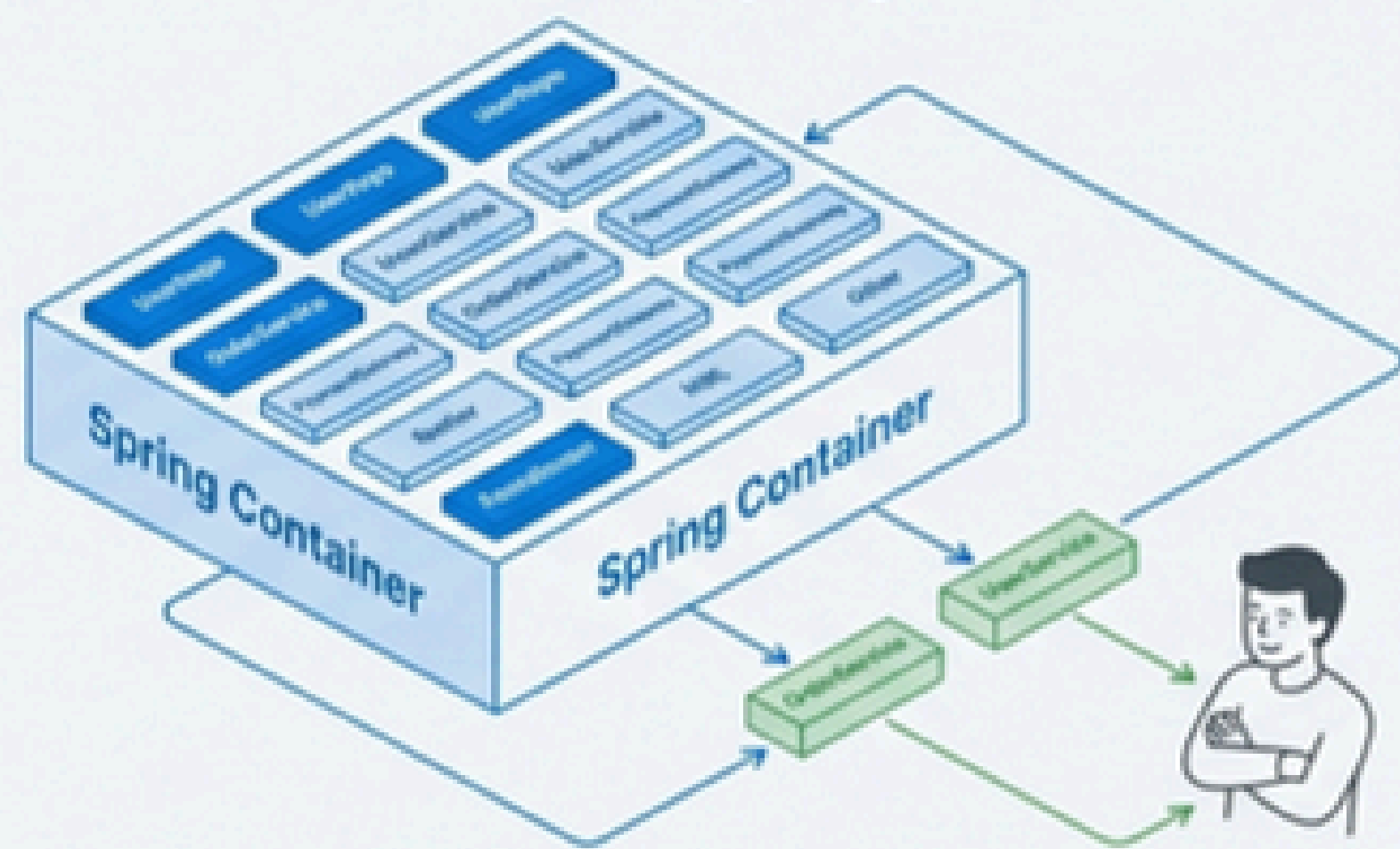
Spring Core의 존재 이유: 개발자는 비즈니스 로직에만 집중하세요

"객체의 생명주기를 대신 관리해 줄 테니,
개발자는 **비즈니스 로직에만 집중**하라."

Before Spring



With Spring



- 스프링의 가장 근본적인 역할은 복잡한 객체 생성, 의존성 설정, 생명주기 관리를 대신 처리해주는 것입니다.
- 개발자는 어떻게 객체를 만들고 연결할지 고민하는 대신, '어떤 기능이 필요한가'에만 집중할 수 있습니다.
- 스프링이 관리하는 이 객체들을 우리는 **Bean**이라고 부릅니다.

Bean을 창고에 등록하는 두 가지 방법

@Component (자동 등록)

- 용도: 내가 직접 작성한 클래스를 빈으로 등록할 때
- 방식: 클래스에 태그(@Component)를 붙여두면, 창고 관리자(@ComponentScan)가 알아서 스캔하여 등록합니다.
- 비유: '이 상자는 부품이니 창고에 보관해주세요'라고 셀프 마킹하는 것.



@Bean (수동 등록)

- 용도: 외부 라이브러리 등 내가 수정할 수 없는 클래스를 빈으로 등록할 때
- 방식: 설정 파일(@Configuration) 안에서 관리자가 직접 객체를 생성하고 등록합니다.
- 비유: 외부에서 배송된 부품을 관리자가 직접 검수하고 라벨을 붙여 창고에 넣는 것.



'@Component': 스프링이 스스로 찾아내게 만드는 마커

위치: 클래스 선언부 위

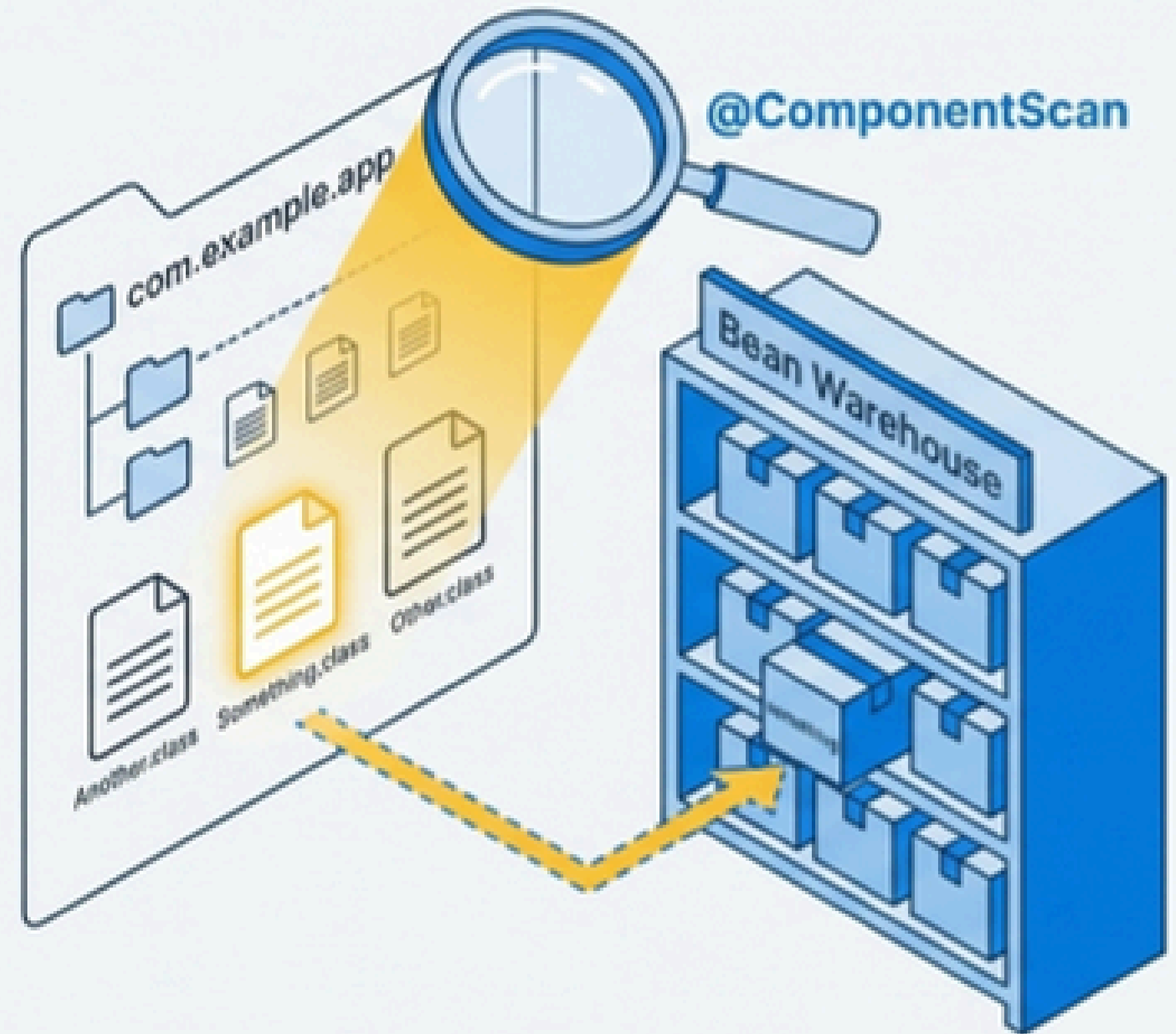
핵심 원리: 스프링 부트가 시작될 때

@ComponentScan이 지정된 패키지 하위를 탐색하며
@Component 어노테이션이 붙은 모든 클래스를 찾아
자동으로 Bean으로 등록합니다.

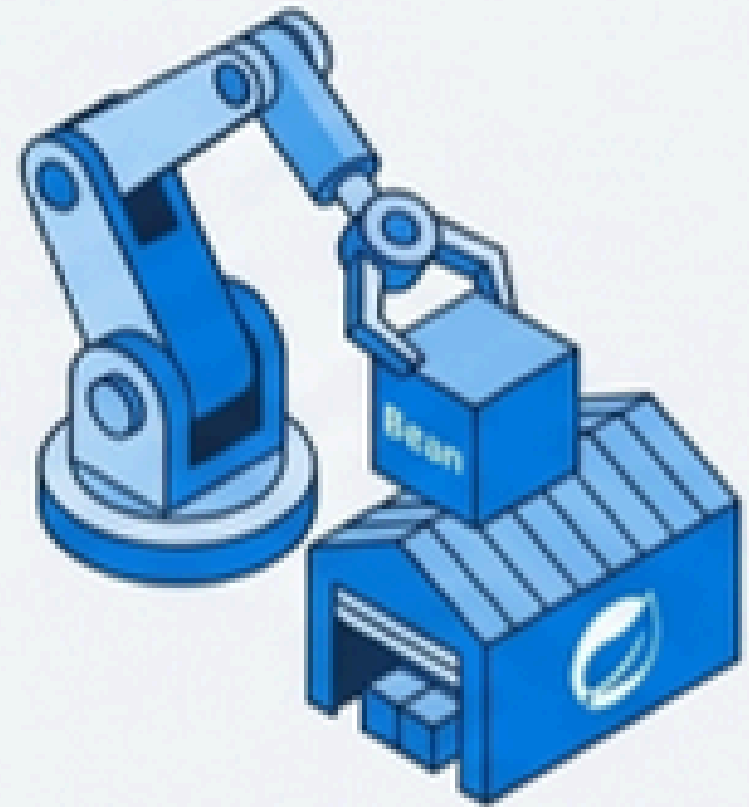
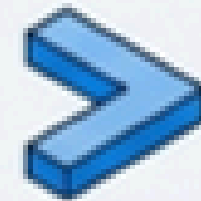
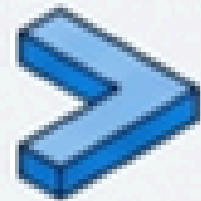
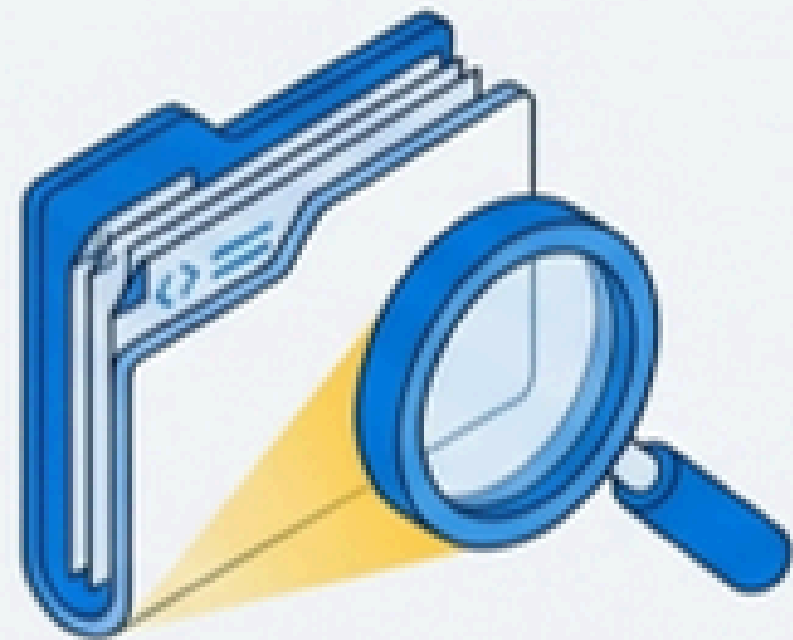
파생 어노테이션: @Controller, @Service,
@Repository는 모두 @Component의 역할을
포함하며, 각 계층에 맞는 추가 기능을 제공합니다.

@Component

```
public class Something {  
    public void doSomething() { ... }  
}
```



`@ComponentScan`의 동작 원리: 3단계 프로세스



1. 탐색 (Scan)

스프링이 지정된 패키지 내 모든 .class 파일을 읽어들이습니다.

2. 검사 (Inspect)

- 리플렉션(Reflection) API를 사용해 각 클래스의 메타정보를 확인합니다.
- @Component (또는 파생 어노테이션)가 붙어있는지 검사합니다.

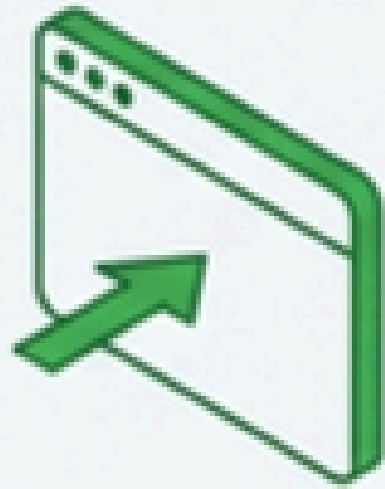
3. 인스턴스화 (Instantiate)

- 대상을 찾으면, new가 아닌 리플렉션 메서드(`Class.getDeclaredConstructor().newInstance()`)를 사용해 객체를 생성합니다.
- 생성된 객체(Bean)를 스프링 컨테이너에 등록합니다.

@Component'의 특별한 가족들: 단순한 이름표 그 이상

@Controller, @Repository, @Service는 왜 구분해서 사용할까요?

각 어노테이션은 계층을 명시할 뿐만 아니라, 스프링이 제공하는 추가 기능을 활성화합니다.



@Controller / @RestController

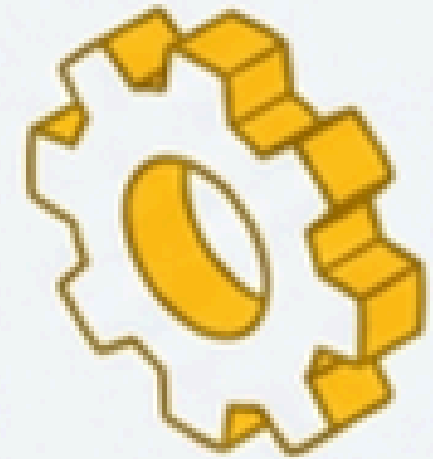
기능: 웹 요청(URL)을 받아 처리하는 클래스임을 명시합니다. 이 어노테이션이 없으면 DispatcherServlet이 컨트롤러로 인식하지 못해 404 에러가 발생합니다.



@Repository

기능: 데이터 접근 계층에서 발생하는 예외를 스프링의 DataAccessException으로 자동 변환해줍니다.

효과: DB 종류(MySQL, Oracle 등)가 바뀌어도 서비스 계층의 예외 처리 코드는 수정할 필요가 없습니다.



@Service

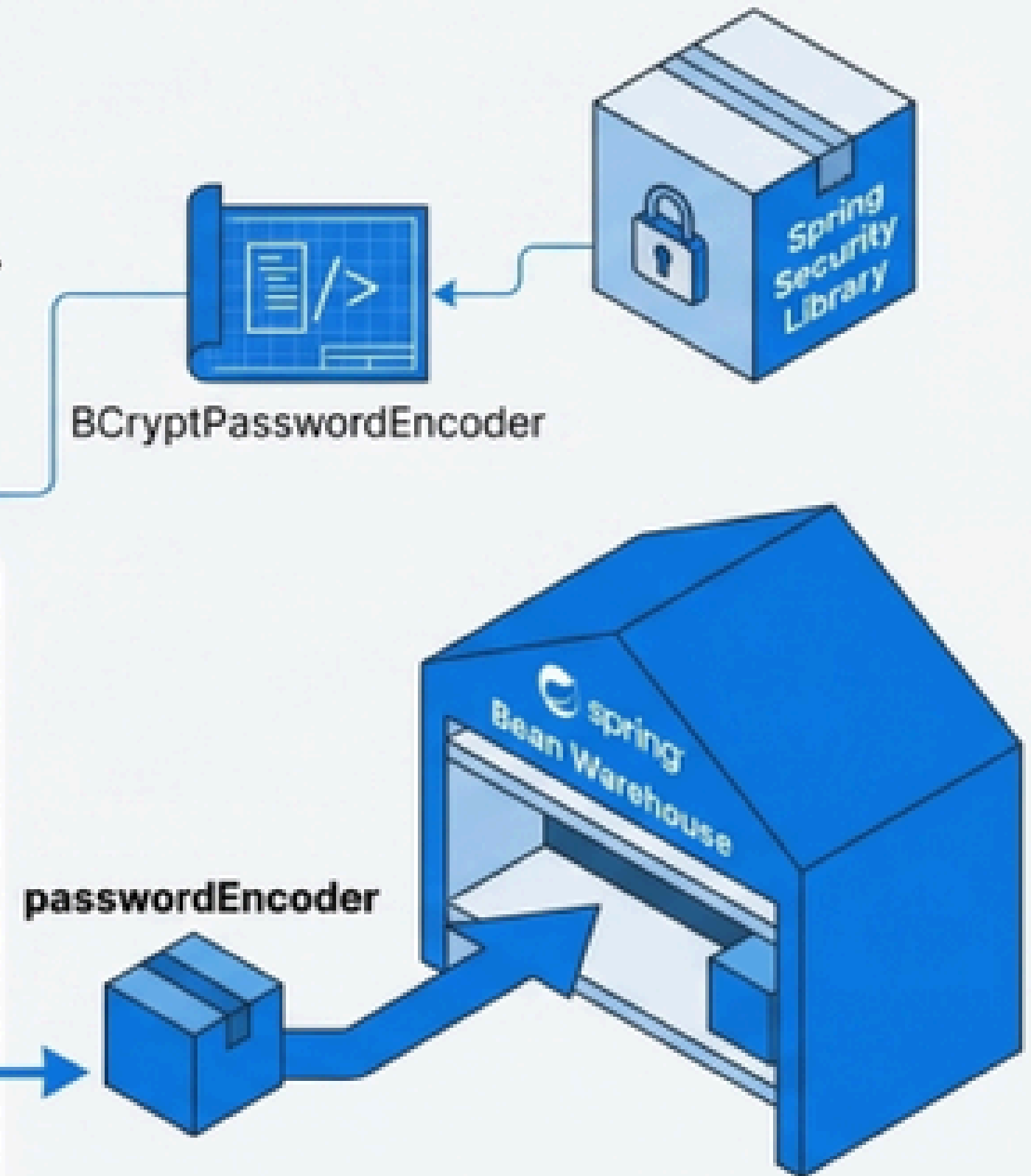
기능: 현재는 특별한 추가 기능이 없지만, 비즈니스 로직의 시작점이자 트랜잭션(@Transactional) 관리의 핵심 단위임을 명시적으로 나타냅니다.

`@Bean`: 외부 라이브러리를 내 창고의 부품으로

- 위치: 메소드 위 (@Configuration 클래스 내부)
- 핵심 용도: 내가 직접 소스 코드를 수정할 수 없는 외부 라이브러리의 클래스나, 복잡한 설정이 필요한 객체를 Bean으로 등록할 때 사용합니다.
- 작동 방식: 개발자가 메소드 본문에서 직접 new를 통해 객체를 생성하고 반환하면, 스프링이 그 결과물을 Bean으로 가져가 관리합니다.

```
// SecurityConfig.java
@Configuration
public class SecurityConfig {

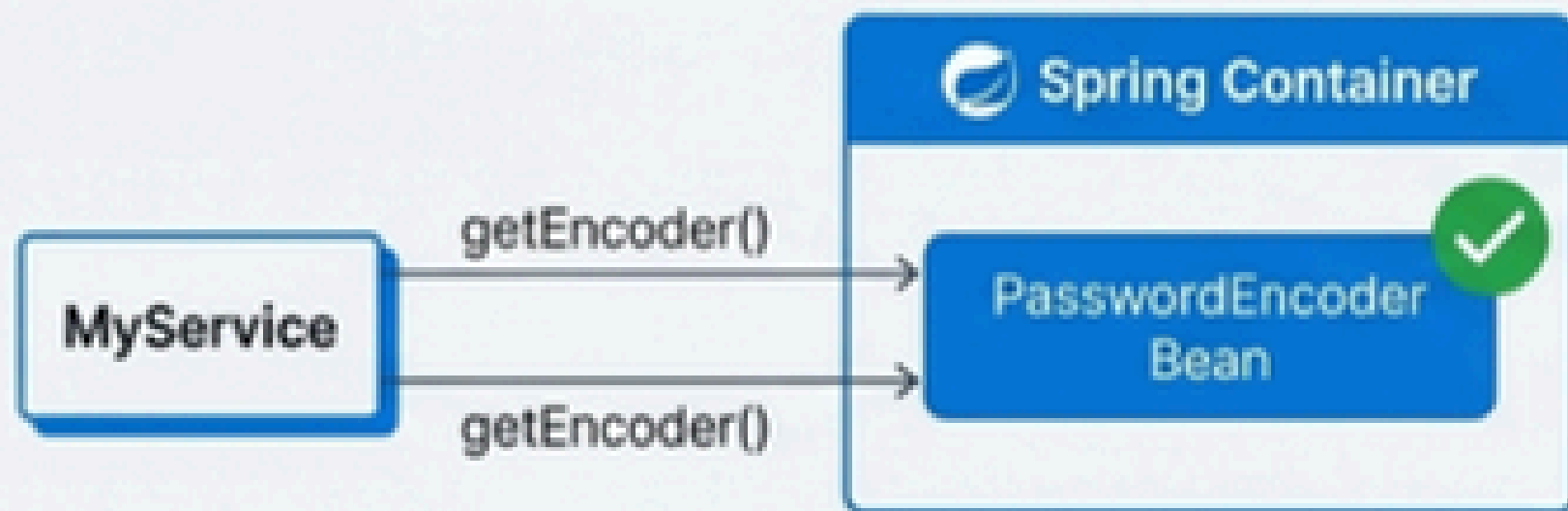
    // BCryptPasswordEncoder는 외부 라이브러리 클래스이므로
    // @Component를 붙일 수 없다.
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```



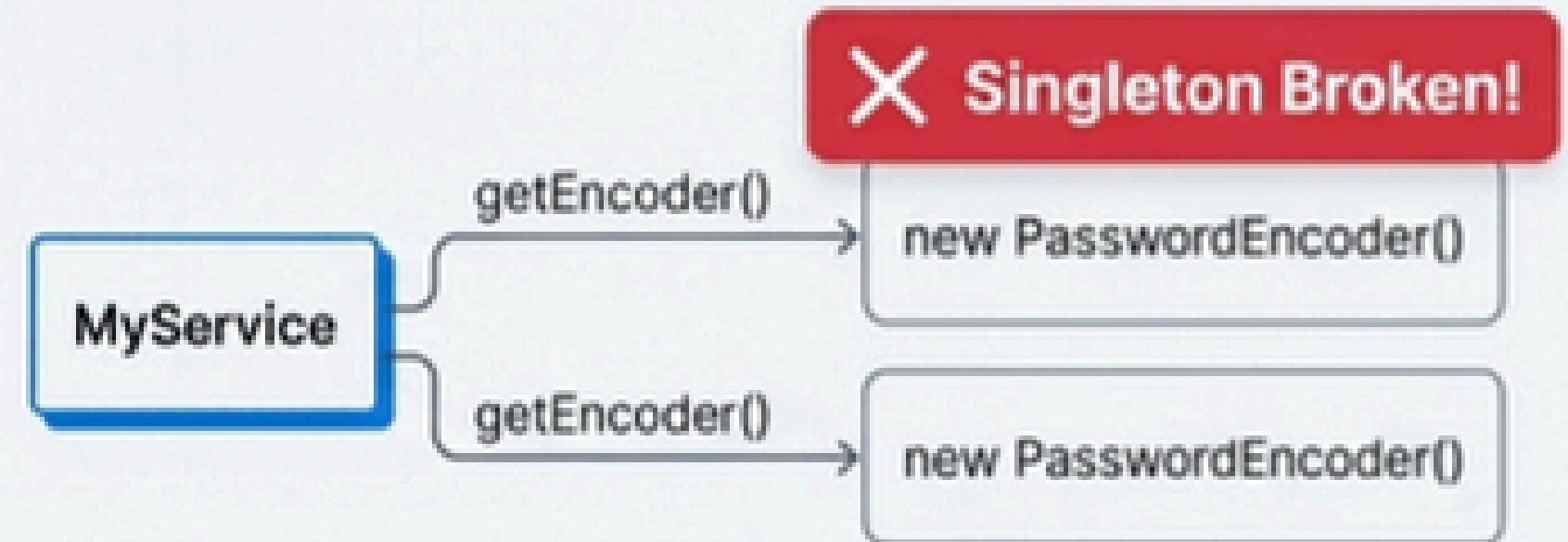
`@Configuration`: 단순한 설정 파일을 넘어 싱글톤을 보장하는 설계자

- 역할 1. 명시: 이 클래스가 Bean 설정을 담고 있는 설정 정보 클래스임을 스프링에게 알립니다. (@Bean 메소드는 보통 이 클래스 내부에 위치)
- 역할 2. 싱글톤 보장: @Configuration 클래스 내에서 @Bean 메소드를 여러 번 호출해도, 스프링은 항상 동일한 단일 객체(싱글톤)를 반환하도록 보장합니다.
- 질문: 어떻게 이런 동작이 가능할까요? 비밀은 **CGLIB** 라이브러리에 있습니다.

With @Configuration



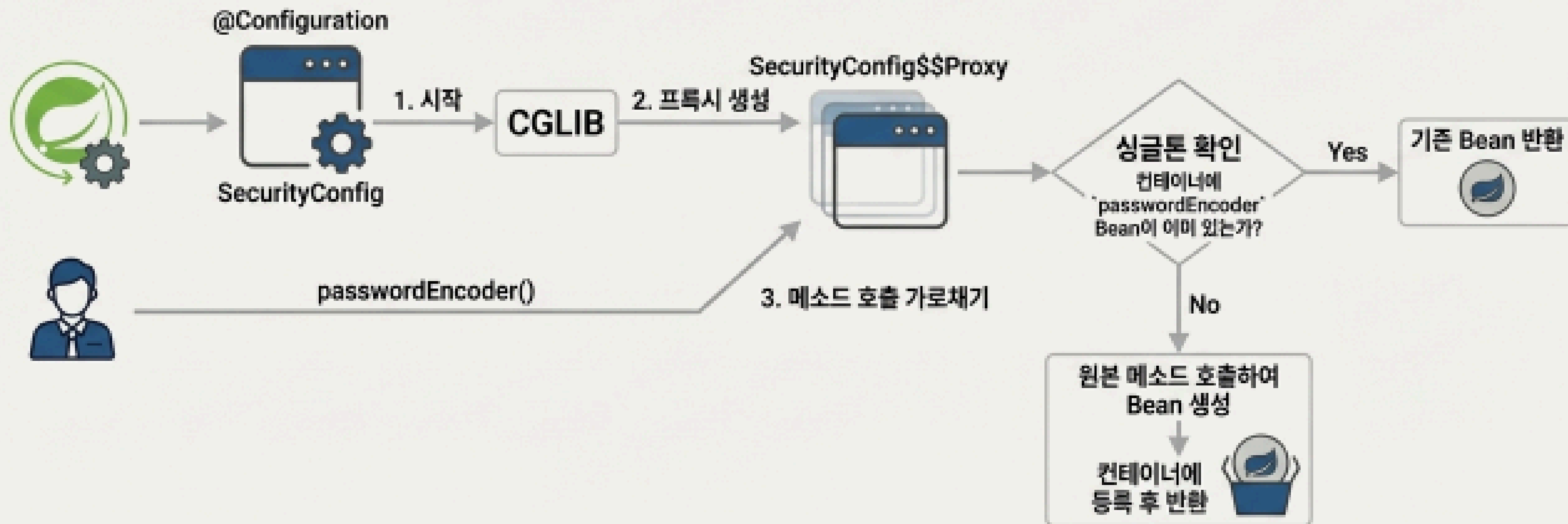
Without @Configuration



공장의 품질 보증: '@Configuration'과 싱글톤

이 클래스가 'Bean 설정 정보'를 담고 있음을 명시하고, Bean의 싱글톤을 보장합니다.

The 'Magic': CGLIB 프록시



Spring Web MVC: 웹 요청을 처리하는 관제탑

Spring Web MVC는 모든 웹 요청을 **DispatcherServlet**이라는 단일 창구가 먼저 받은 뒤, 가장 적절한 담당자(Controller)에게 작업을 분배하고, 그 결과를 모아 최종 응답을 만들어주는 정교한 프레임워크입니다.

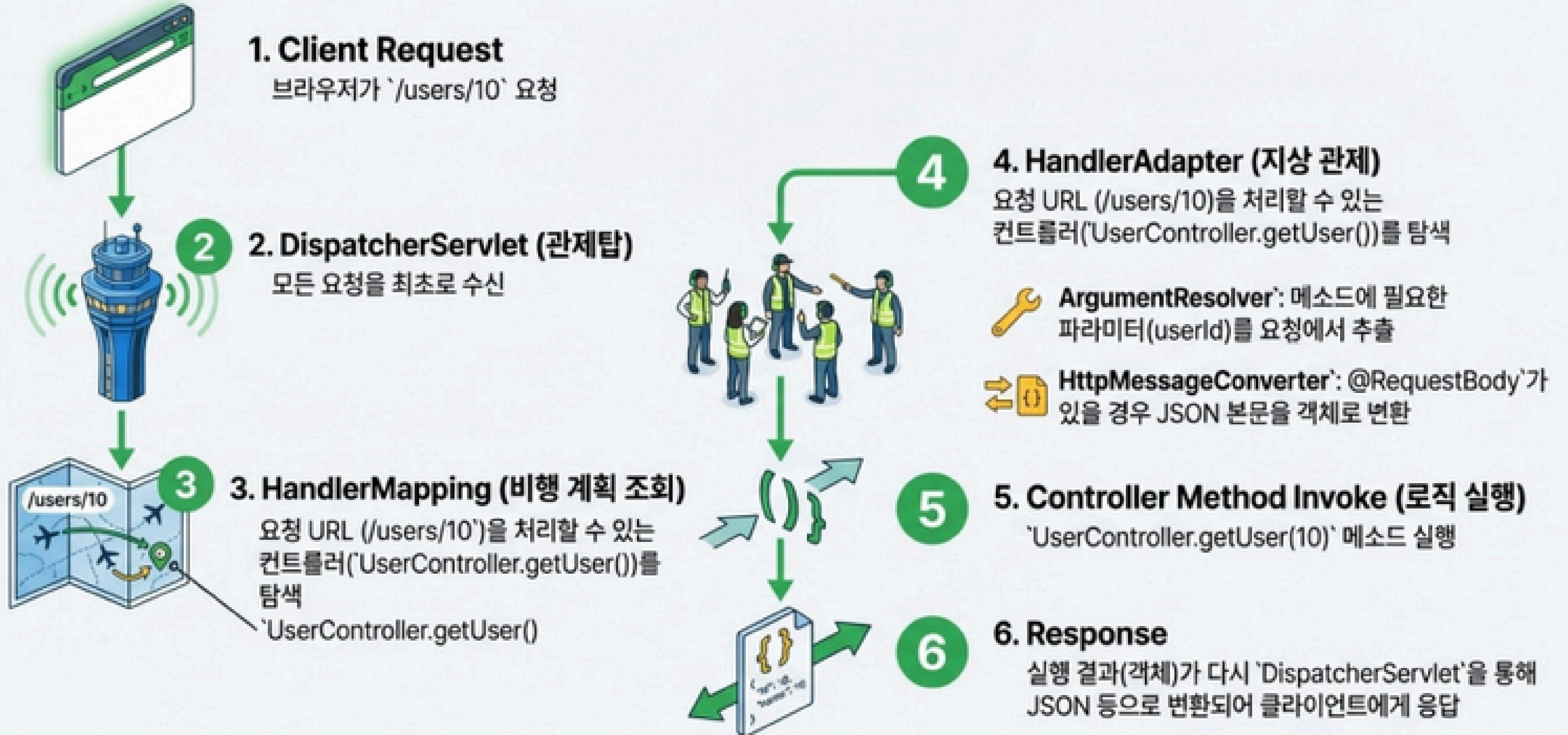
'DispatcherServlet': 모든 항공기(HTTP Request)의 이착륙을 통제하는 관제탑.

'HandlerMapping': 들어온 항공편의 목적지('URL')를 보고 어느 활주로(@Controller)로 보내야 할지 결정하는 비행 계획.

'Controller': 실제 승객을 처리하는 활주로 및 터미널.



하나의 웹 요청이 처리되기까지의 여정

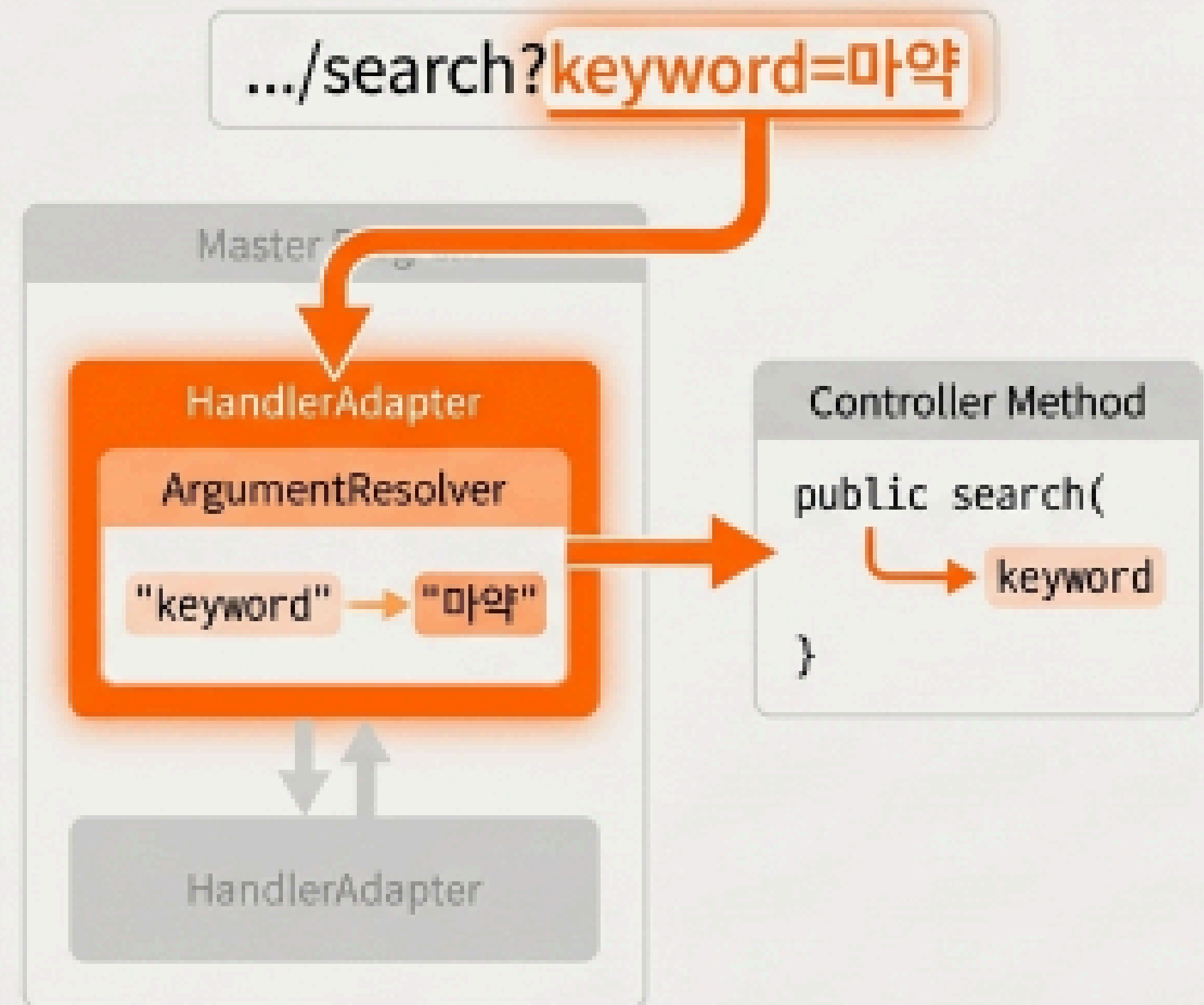


요청에서 데이터 낚아채기: @RequestParam

URL의 쿼리 스트링(?key=value)에서 특정 값을 낚개로 추출할 때 사용.

```
@GetMapping("/search")  
public String search(  
    @RequestParam(value = "keyword") String keyword  
) { ... }
```

Request: GET /search?keyword=마약

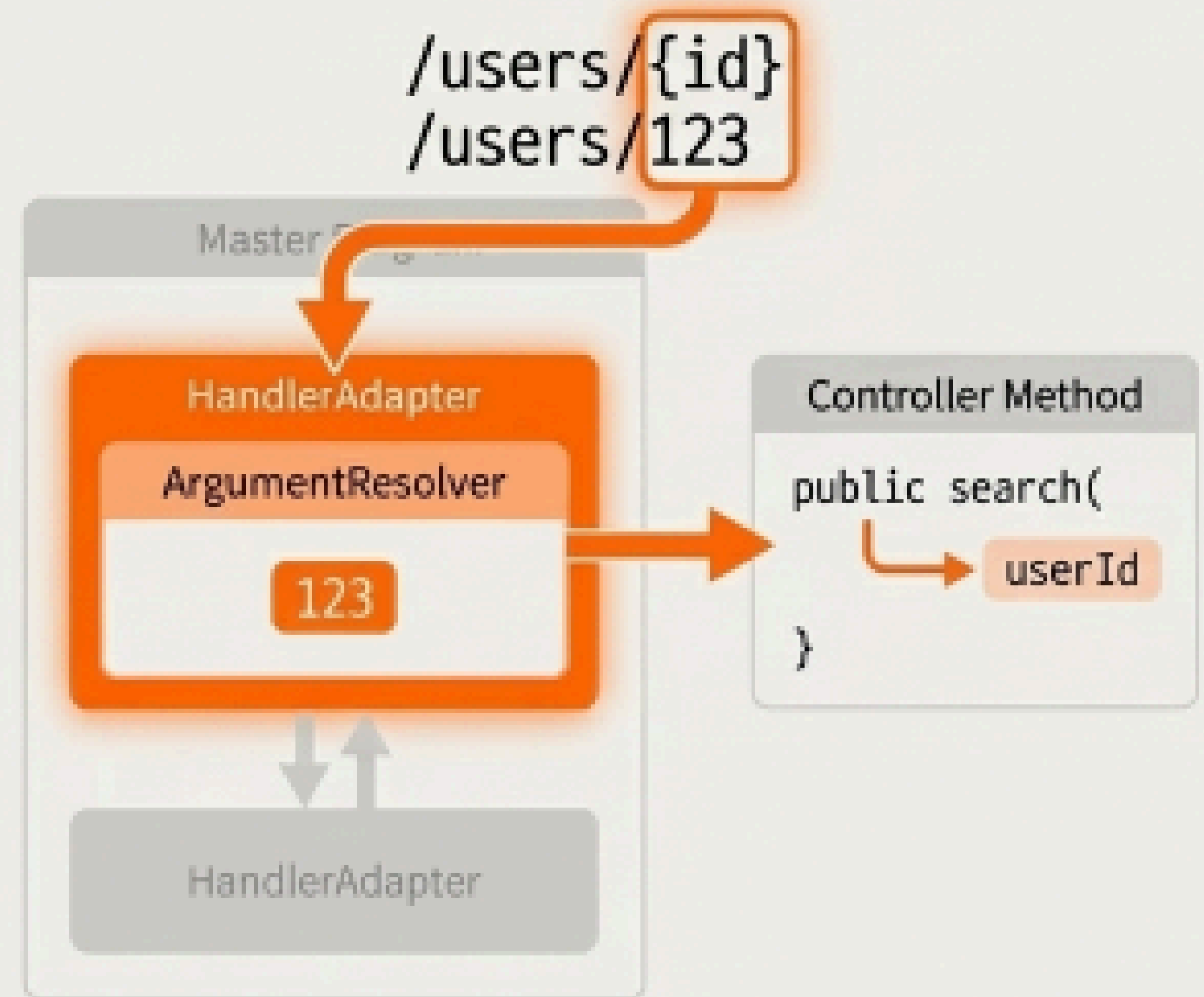


URL 경로 자체를 데이터로: @PathVariable

RESTful API에서 리소스의 고유 ID처럼,
URL 경로의 일부를 변수로 사용할 때.

```
@GetMapping("/users/{id}")  
public String getUser(  
    @PathVariable("id") Long userId  
) { ... }
```

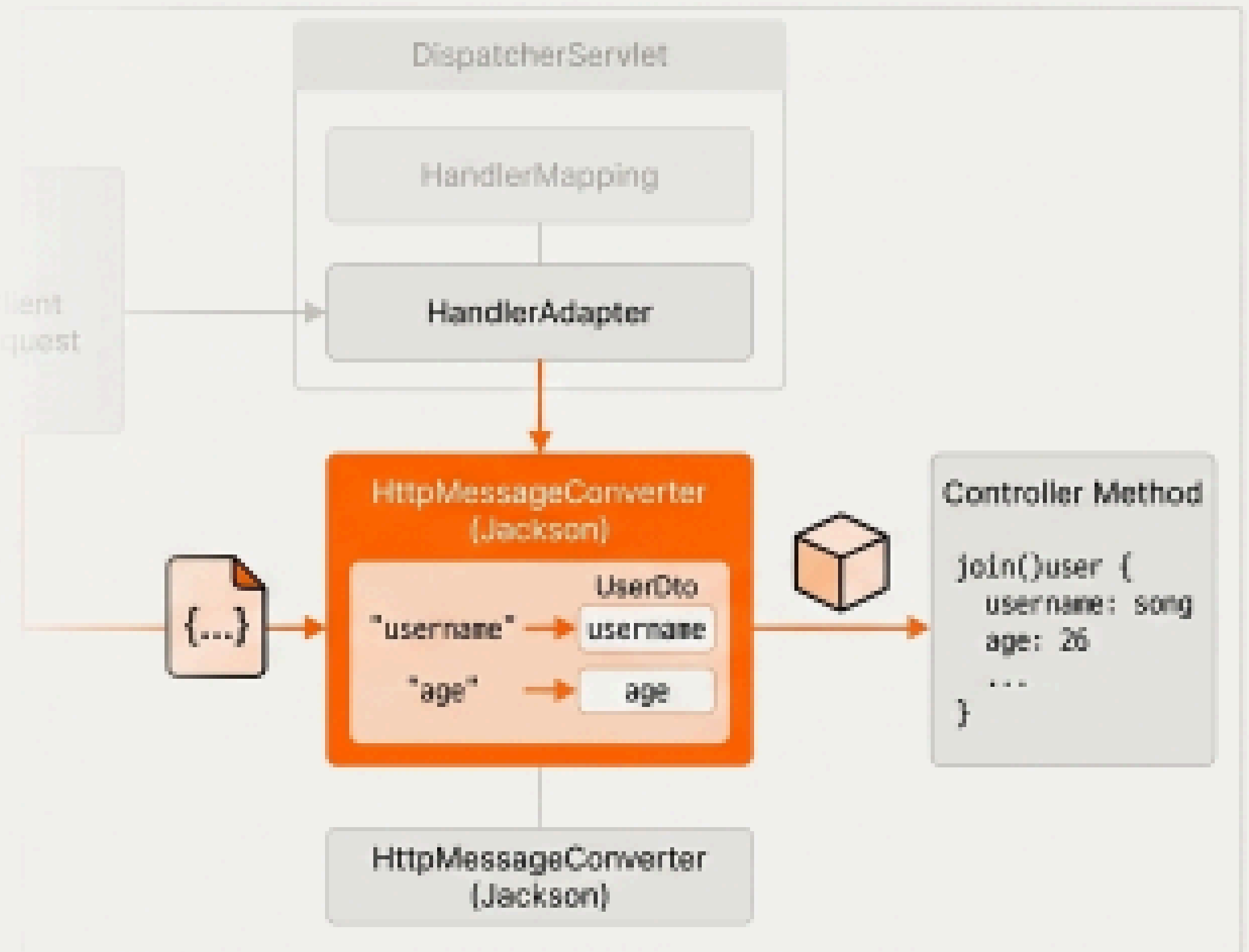
Request: GET /users/123



요청 본문(JSON) 통째로 변환하기: @RequestBody

HTTP 요청의 본문(Body)에 담긴 JSON 데이터를 자바 객체로 변환.

```
Request Body: {  
  "username": "song",  
  "age": 26  
}  
  
@PostMapping("/users")  
public void join(@RequestBody UserDto user) {  
  ...  
}
```

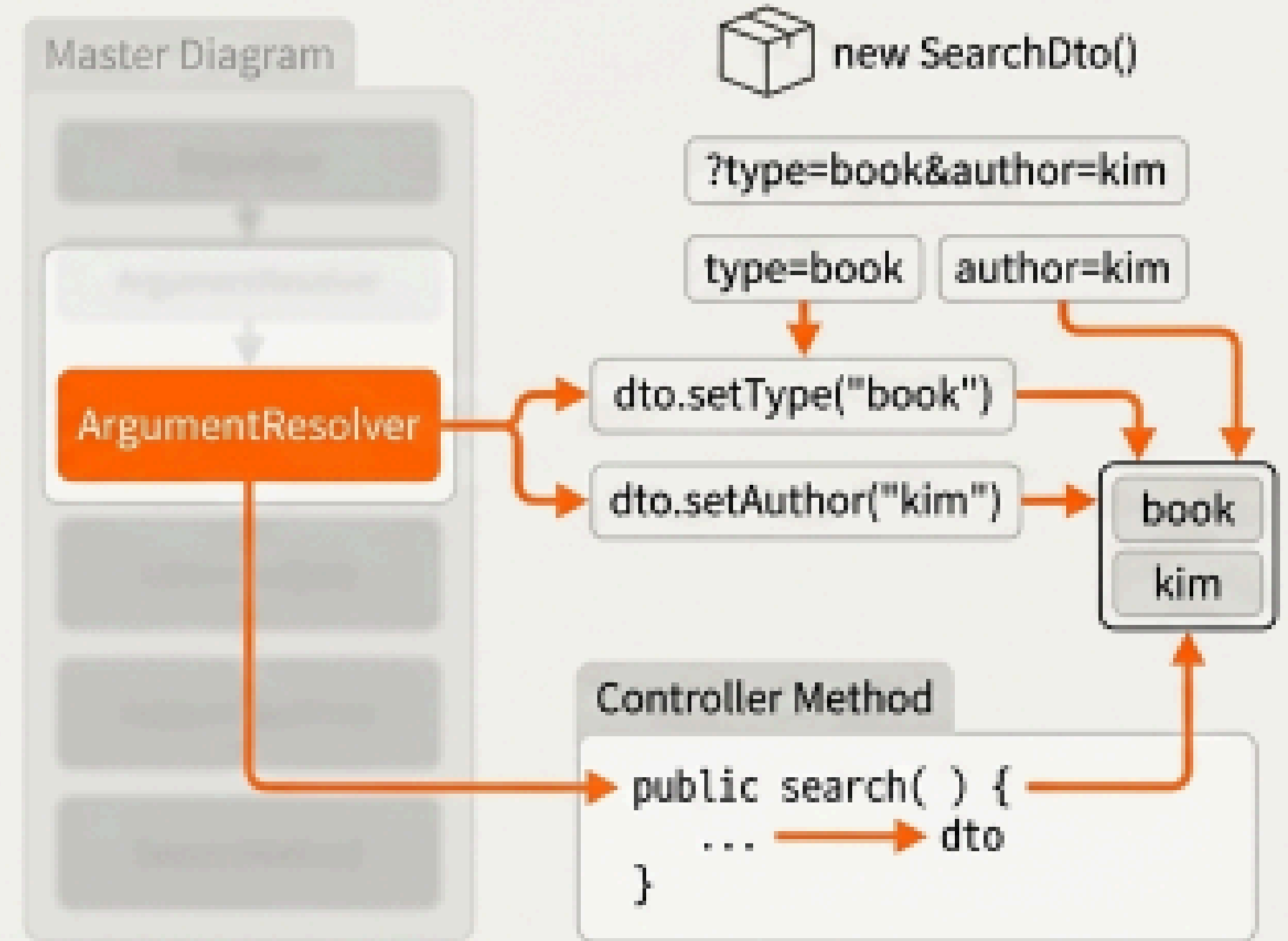


파라미터 묶어서 받기: @ModelAttribute

여러 개의 쿼리 파라미터나
HTML <form> 데이터를
하나의 객체(DTO)로 바인딩.

JSON 데이터는 처리할 수 없습니다.

```
// Request: GET /filter?type=book&author=kim  
  
@GetMapping("/filter")  
public String filter(@ModelAttribute SearchDto dto) { ...}
```



'@ResponseBody' & @RestController: 자바 객체를 JSON 응답으로

@ResponseBody

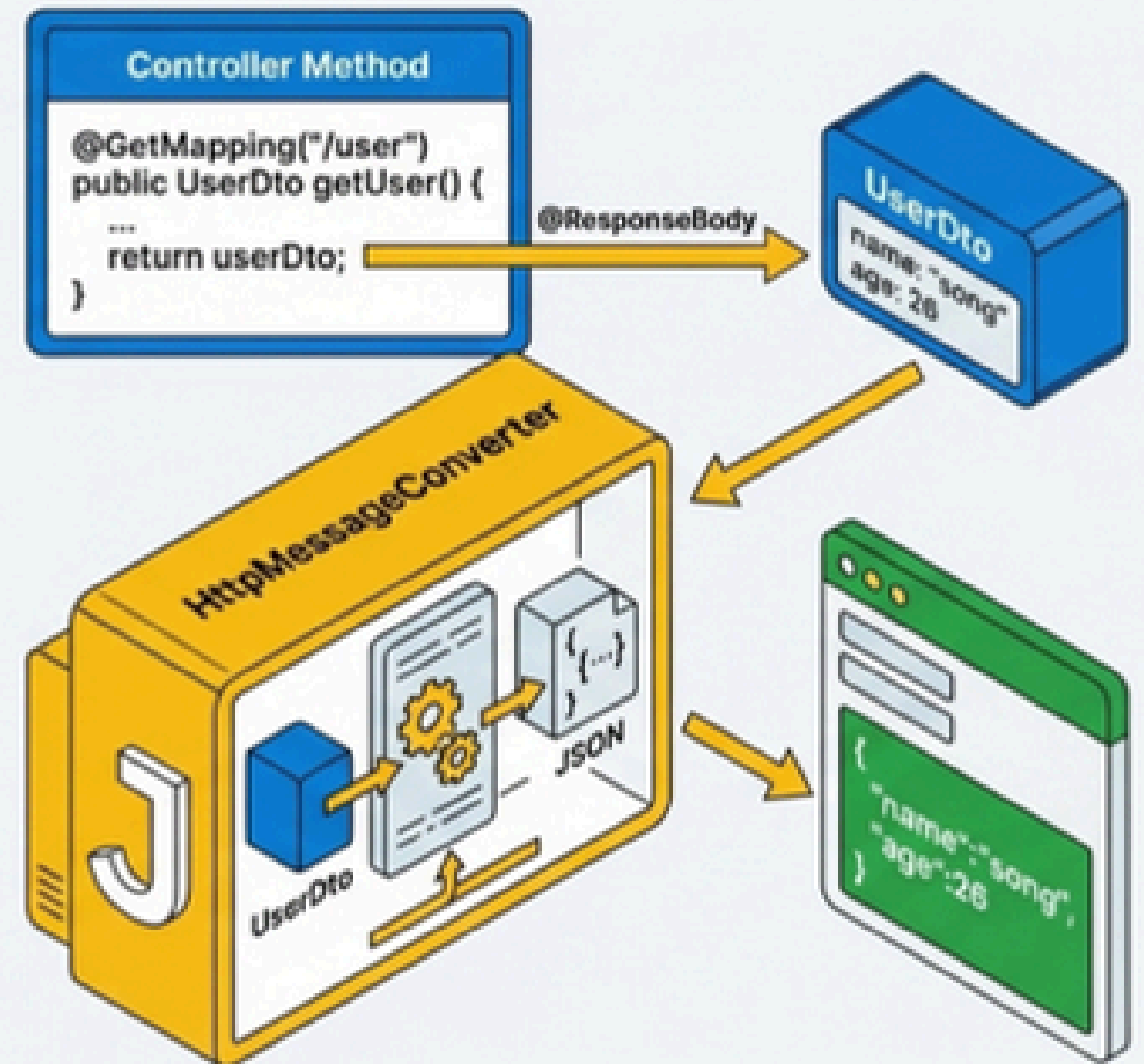
역할: 컨트롤러 메소드의 리턴값을 View 이름으로 해석하지 말고, HTTP 응답 본문(Body)에 직접 쓰라는 지시.

동작: 리턴된 자바 객체(DTO, Map, List 등)는 `HttpMessageConverter(Jackson)`에 의해 자동으로 JSON 문자열로 변환됩니다.

@RestController

정의: `@Controller` + `@ResponseBody`의 조합.

효과: 클래스 레벨에 붙이면, 해당 클래스의 모든 메소드에 `@ResponseBody`가 적용된 것처럼 동작합니다. API 서버를 만들 때 필수적입니다.



한눈에 보는 비교: `@RequestBody` vs `@ModelAttribute`

@RequestBody

데이터 형식: JSON

데이터 위치: HTTP Request Body

동작 주체: HttpMessageConverter

핵심 원리: JSON 라이브러리(Jackson)가 파싱 후 객체 필드에 주입

POST /api/members

Content-Type: application/json

```
{ "username": "song", "age": 26 }
```

@ModelAttribute

데이터 형식: Query String 또는 Form Data (x-www-form-urlencoded)

데이터 위치: URL 또는 HTTP Request Body (Form)

동작 주체: ArgumentResolver

핵심 원리: 파라미터 이름과 DTO 프로퍼티 이름을 매칭하여 Setter로 주입

POST /members/new

Content-Type: application/x-www-form-urlencoded

username=song&age=26

언제 무엇을 쓸까?: @PathVariable vs @RequestParam

"값이 없으면 아예 다른 페이지가 되는가?"

'@PathVariable': YES

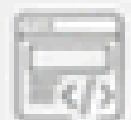
- 의미: 리소스(엔터티)를 식별하는 핵심 정보.
- 예시: /posts/123 vs /posts/124. 123이 없으면 어떤 게시물인지 식별 불가능.
- 용도: 특정 게시물 조회, 특정 회원 정보 수정 등.

'@RequestParam': NO

- 의미: 정렬, 필터링, 검색 등 부가적인 정보.
- 예시: /posts?page=2, /posts?sort=newest. page=2가 없어도 게시물 목록이라는 핵심 페이지는 동일.
- 용도: 검색어, 페이지 번호, 정렬 기준 등.

Real-World Code Example

```
// postId는 핵심 식별자, content/topicId는 부가 정보
public ApiResult updatePost(
    @PathVariable Long postId,    // 이 게시글을
    @RequestParam String content, // 이 내용으로
    @RequestParam Long topicId    // 이 토픽으로 수정
) { ... }
// 예상 요청: /api/posts/10?content=수정내용&topicId=5
```



쿼리 파라미터 처리 방식의 대결: @RequestParam vs @ModelAttribute

같은 URL 요청을 서버에서 어떻게 받을 것인가의 차이

GET /members?username=song&age=26

@RequestParam
(날개로 받기 - One by One)

한두 개의 파라미터를 받을 때

```
public void getMember(  
    @RequestParam String username, // 1:1 매핑  
    @RequestParam int age         // 1:1 매핑  
) { ... }  
}
```

1:1 매핑

@ModelAttribute

(객체로 묶어 받기 - Bundled as an Object)

검색 조건 등 파라미터가 많을 때

```
public void getMember(  
    @ModelAttribute MemberDto dto  
) {  
    // dto.getUsername(), dto.getAge()  
}
```

1:1 매핑이 필요한가, 아니면 Many:1 매핑이 효율적인가?